

Class Activity CO3

Sameer Ahmed G

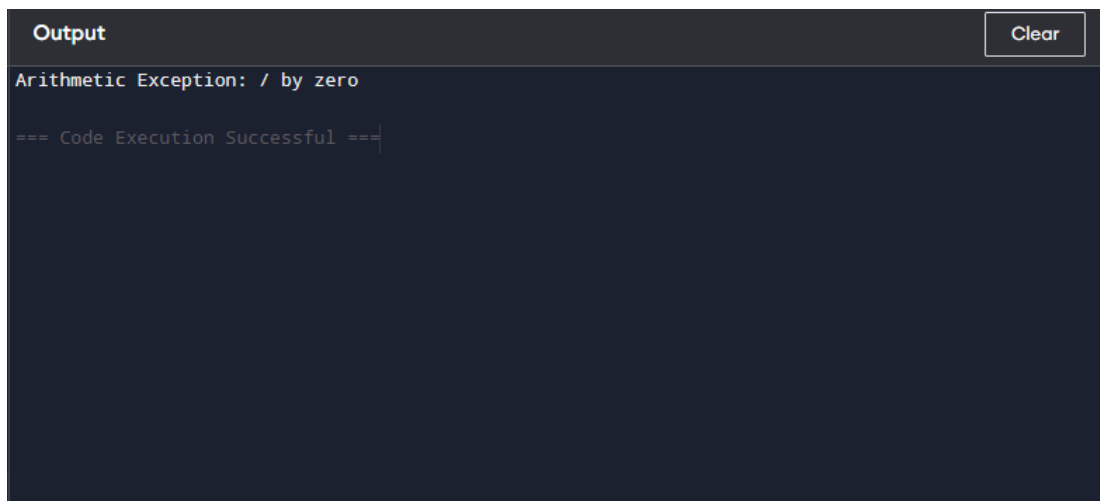
192421154

1. Multiple Catch Statements

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
            int[] x = {1, 2};  
            System.out.println(x[5]);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Arithmetic Exception: " + e.getMessage());  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Exception: " + e.getMessage());  
        }  
    }  
}
```

Output:

Arithmetic Exception: / by zero



The screenshot shows a dark-themed IDE output window. At the top, there is a tab labeled 'Output' and a 'Clear' button. The output text is as follows:

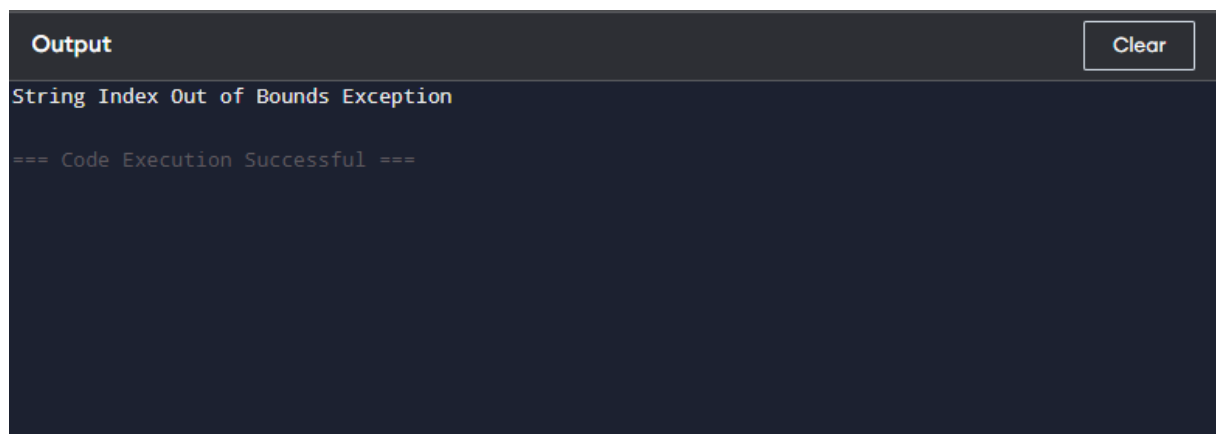
```
Arithmetic Exception: / by zero  
  
=== Code Execution Successful ===
```

2. StringIndexOutOfBoundsException

```
public class Main {  
    public static void main(String[] args) {  
        String s = "Java";  
  
        try {  
            System.out.println(s.charAt(10));  
        }  
        catch (StringIndexOutOfBoundsException e) {  
            System.out.println("String Index Out of Bounds Exception");  
        }  
    }  
}
```

Output:

String Index Out of Bounds Exception

A screenshot of a code execution environment's output window. The window has a dark background. At the top left, the word "Output" is written in a light color. At the top right, there is a button labeled "Clear". The main area of the window displays the text "String Index Out of Bounds Exception" in a light color. Below this text, there is a line of text that reads "=== Code Execution Successful ===".

```
Output Clear  
String Index Out of Bounds Exception  
=== Code Execution Successful ===
```

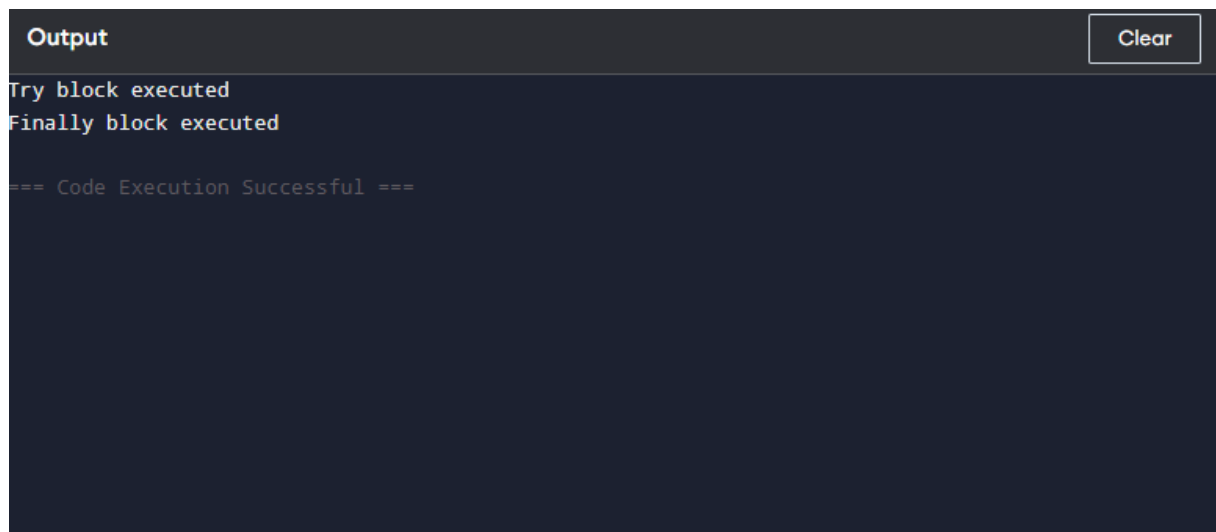
3. finally

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Try block executed");  
        }  
        finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

Output:

Try block executed

Finally block executed



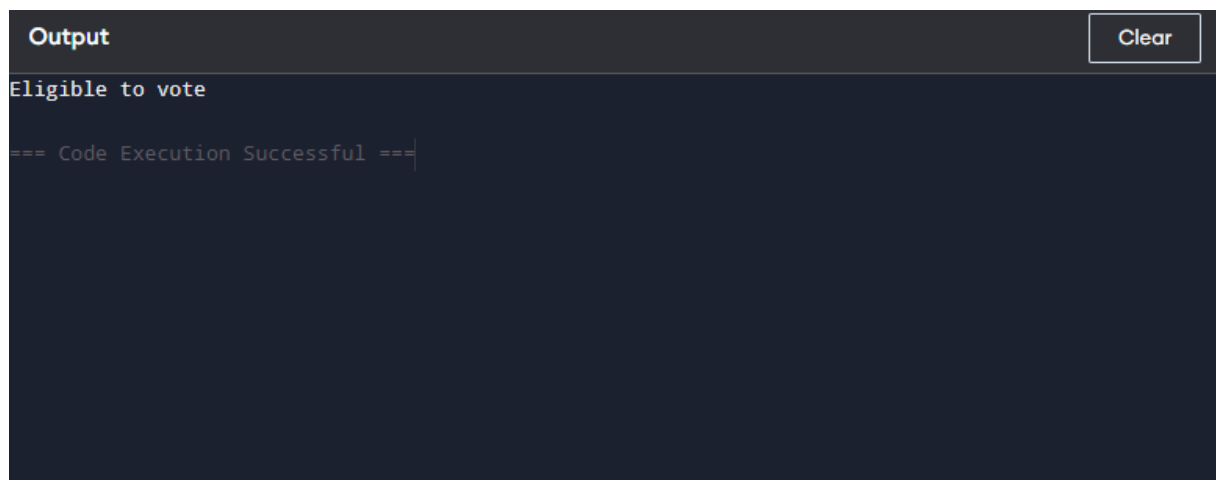
The screenshot shows a dark-themed IDE output window. At the top, there is a header bar with the word "Output" on the left and a "Clear" button on the right. Below the header, the output text is displayed in a monospaced font. It shows "Try block executed" followed by "Finally block executed" on the next line. At the bottom of the output, there is a line of text: "=== Code Execution Successful ===".

4. throw – Voting Eligibility

```
public class Main {  
  
    static void checkAge(int age) {  
        if (age < 18)  
            throw new ArithmeticException("Not eligible to vote");  
  
        System.out.println("Eligible to vote");  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(20);  
        }  
        catch (ArithmeticException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output:

Eligible to vote

A screenshot of a code execution output window. The window has a dark background. At the top, there is a header bar with the word "Output" on the left and a "Clear" button on the right. Below the header, the text "Eligible to vote" is displayed. At the bottom, there is a line of text that reads "=== Code Execution Successful ===".

```
OutputClear  
Eligible to vote  
=== Code Execution Successful ===
```

5. throws

```
import java.io.*;
```

```
public class Main {
```

```
    static void readFile() throws IOException {
```

```
        FileReader f = new FileReader("sample.txt");
```

```
        f.close();
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            readFile();
```

```
            System.out.println("File opened successfully");
```

```
        }
```

```
        catch (IOException e) {
```

```
            System.out.println("IOException: " + e.getMessage());
```

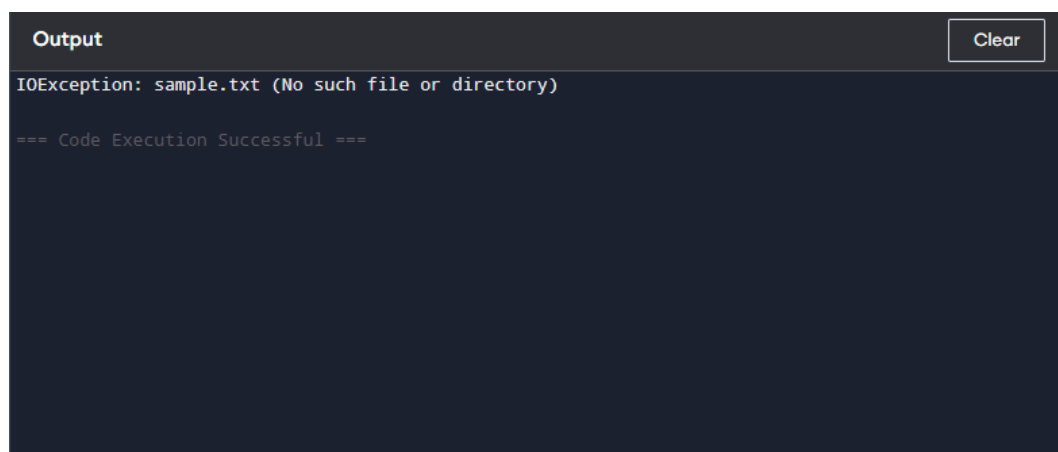
```
        }
```

```
    }
```

```
}
```

Output:

IOException: sample.txt (No such file or directory)

A screenshot of a code execution environment's output window. The window has a dark background. At the top left, the word "Output" is written in a light color. At the top right, there is a button labeled "Clear". The main area of the window displays the output of the program: "IOException: sample.txt (No such file or directory)" on the first line, followed by "=== Code Execution Successful ===" on the second line.

```
Output
```

```
IOException: sample.txt (No such file or directory)
```

```
=== Code Execution Successful ===
```